به نام خدا

موضوع پژوهش:

تشخیص احساس از روی متن فارسی با استفاده از رویکردهای ماشین لرنینگ کلاسیک و یادگیری عمیق

نگارش :**علی نیکوخصال**
استاد راهنما :**دکتر مینا مسعودی‌فر**

شهریور ۱۴۰۵

فهرست مطالب

فهرست مطالب ۲

مقدمه ۱۰ ..... ٤

اهداف پروژه ۱.۱ ..... ٤

ساختار گزارش ۱.۲ ..... ٥

مبانی نظری ۲۰ ..... ٦

مفاهیم پایه‌ی یادگیری ماشین ۲.۱ ..... ٦

مدل‌های ماشین لرنینگ کلاسیک برای متن ۲.۲ ..... ٦

GRU و LSTM ساده تا RNN از ۲.۳ ..... ۷

نرخ یادگیری، تابع هزینه و بهینه‌ساز ۲.۴ ..... ۸

پردازش زبان طبیعی فارسی ۲.۵ ..... ۸

دیتاست‌ها ۳۰ ..... ۱۰

(دیتاست اصلی) ArmanEmo ۳.۱ ..... ۱۰

(دیتاست مکمل برای آزمایش افزایش داده) EmoPars ۳.۲ ..... ۱۱

پاک‌سازی و پیش‌پردازش داده ۳.۳ ..... ۱۱

روش‌شناسی و پیاده‌سازی ۴۰ ..... Error! Bookmark not defined.

معماری کلی سامانه ۴.۱ ..... ۱۲

مدل‌های کلاسیک ۴.۲ ..... ۱۲

مدل‌های یادگیری عمیق ۴.۳ ..... ۱۲

رابط کاربری ۴.۴ ..... ۱۳

آزمایش‌ها و نتایج ۵۰ ..... ۱٤

نتایج پایه :مدل‌های کلاسیک در برابر مدل‌های عمیق ۵.۱ ..... ۱٤

آزمایش اول :بررسی علت ضعف مدل عمیق اولیه ۵.۲ ..... ۱٤

آزمایش دوم :آیا افزایش داده با EmoPars کمک می‌کند؟ ۵.۳ ..... ۱٥

آزمایش سوم :آیا ساده‌سازی معماری کمک می‌کند؟ ۵.۴ ..... ۱٥

آزمایش چهارم :آیا افزودن ویژگی سطح کاراکتر کمک می‌کند؟ ۵.۵ ..... ۱٦

چرا درصد اطمینان مدل گاهی پایین به‌نظر می‌رسد؟ ۵.۶ ..... ۱۷

نمونه‌ی کیفی :کلمات خارج از دایره واژگان آموزشی ۵.۷ ۱۸
جمع‌بندی آزمایش‌ها ۵.۸ ۱۸
بحث و تحلیل ۶. ۱۹
نتیجه‌گیری و کارهای آینده ۷. ۲۰
منابع ۲۱

۱. مقدمه

با گسترش شبکه‌های اجتماعی و فروشگاه‌های آنلاین، حجم عظیمی از متن فارسی روزانه توسط کاربران تولید می‌شود که در پس آن احساسات گوناگونی مانند خشم، شادی، غم یا نفرت نهفته است .تشخیص خودکار این احساسات کاربردهای متعددی دارد؛ از تحلیل نظرات مشتریان و مدیریت شبکه‌های اجتماعی گرفته تا سامانه‌های پشتیبانی هوشمند .هدف این پروژه، طراحی و پیاده‌سازی یک سامانه‌ی کامل برای تشخیص احساس از روی متن فارسی است که متن ورودی کاربر را به یکی از هفت دسته‌ی خشم، ترس، شادی، نفرت، غم، تعجب یا سایر طبقه‌بندی می‌کند.

بر خلاف زبان انگلیسی که منابع و ابزارهای پردازش زبان طبیعی فراوانی برای آن وجود دارد، زبان فارسی از نظر دیتاست‌های برچسب‌گذاری‌شده و ابزارهای پیش‌پردازش محدودیت‌های خاص خودش را دارد؛ نیم‌فاصله، صرف پیچیده‌ی افعال، و نبود فاصله‌ی استاندارد بین کلمات از جمله چالش‌های زبان فارسی در پردازش خودکار متن هستند .این پروژه علاوه بر پیاده‌سازی مدل‌های طبقه‌بندی، به این چالش‌های خاص زبان فارسی نیز می‌پردازد.

۱.۱ اهداف پروژه

- آشنایی عملی با مفاهیم پایه‌ی یادگیری ماشین)تابع هزینه، نرخ یادگیری، بهینه‌ساز (و پیاده‌سازی آن‌ها
- مقایسه‌ی رویکرد ماشین لرنینگ کلاسیک (TF-IDF) با رویکرد یادگیری عمیق (GRUوLSTM) روی یک مسئله‌ی واقعی
- پیاده‌سازی پایپلاین کامل پیش‌پردازش متن فارسی با استفاده از ابزارهای تخصصی این زبان
- طراحی یک رابط کاربری تعاملی برای استفاده‌ی عملی از مدل‌های آموزش‌دیده
- انجام آزمایش‌های کنترل‌شده برای بررسی روش‌های بهبود عملکرد)افزایش داده، تنظیم معماری(و تحلیل صادقانه‌ی نتایج، حتی در مواردی که فرضیه‌ی اولیه رد می‌شود

۱.۲ ساختار گزارش

در فصل دوم، مبانی نظری لازم شامل مدل‌های کلاسیک متن، شبکه‌های عصبی بازگشتی (LSTM/GRU) و مفاهیم پایه‌ی بهینه‌سازی مرور می‌شود. فصل سوم به معرفی دیتاست‌های استفاده‌شده اختصاص دارد. فصل چهارم روش پیاده‌سازی و معماری سامانه را شرح می‌دهد. فصل پنجم نتایج آزمایش‌ها را به‌همراه سه آزمایش کنترل‌شده برای بهبود مدل عمیق ارائه می‌کند. فصل ششم به بحث و تحلیل نتایج و فصل هفتم به نتیجه‌گیری و پیشنهاد کارهای آینده می‌پردازد.

۲. مبانی نظری

۲.۱ مفاهیم پایه‌ی یادگیری ماشین

در یادگیری ماشین نظارت‌شده(Supervised Learning) ، مدل با دیدن نمونه‌های برچسب‌خورده (متن و احساس متناظر آن) یاد می‌گیرد یک الگو را از داده استخراج کند. برای این منظور، داده به سه بخش تقسیم می‌شود: داده‌ی آموزش (Train) که مدل روی آن یاد می‌گیرد، داده‌ی اعتبارسنجی (Validation) که برای تنظیم ابرپارامترها و جلوگیری از بیش‌برازش استفاده می‌شود، و داده‌ی آزمون (Test) که فقط یک‌بار و در پایان برای سنجش بی‌طرفانه‌ی عملکرد نهایی به‌کار می‌رود.

دو خطای رایج در این فرایند عبارتند از بیش‌برازش(Overfitting) ، یعنی زمانی که مدل به‌جای یادگیری الگوی کلی، جزئیات و نویز داده‌ی آموزش را حفظ می‌کند و روی داده‌ی جدید ضعیف عمل می‌کند؛ و کم‌برازش(Underfitting) ، یعنی زمانی که مدل حتی روی داده‌ی آموزش هم عملکرد ضعیفی دارد چون ظرفیت کافی برای یادگیری الگو را ندارد. در این پروژه، بیش‌برازش مدل‌های عمیق یکی از چالش‌های اصلی مشاهده‌شده بود که در فصل پنجم به‌تفصیل بررسی می‌شود.

از آنجا که دیتاست این پروژه از نظر تعداد نمونه در هر کلاس نامتوازن است (برای مثال کلاس «سایر» بسیار پرتکرارتر از «نفرت» است)، معیار Accuracy به‌تنهایی گمراه‌کننده است؛ چون مدلی که همیشه پرتکرارترین کلاس را پیش‌بینی کند هم Accuracy بالایی می‌گیرد. به همین دلیل در این پروژه از معیار F۱-Macroاستفاده شده که میانگین F۱-Score تک‌تک کلاس‌ها را (بدون وزن‌دهی بر اساس فراوانی) محاسبه می‌کند و عملکرد مدل روی کلاس‌های کم‌تکرار را هم منصفانه منعکس می‌کند.

۲.۲ مدل‌های ماشین لرنینگ کلاسیک برای متن

پیش از استفاده از شبکه‌های عصبی، باید متن را به بردار عددی تبدیل کرد. روش TF-IDF (Term Frequency–Inverse Document Frequency) به هر کلمه در هر سند وزنی می‌دهد که متناسب با تکرار آن کلمه در همان سند و نسبت عکس با فراوانی آن در کل دیتاست است؛ به این ترتیب کلماتی که در

یک سند خاص پرتکرارند ولی در کل دیتاست نادرند (و بنابراین تمایزدهنده‌اند)، وزن بیشتری می‌گیرند. این روش ترتیب کلمات را در نظر نمی‌گیرد(رویکردBag-of-Words) اما به‌عنوان یک Baseline بسیار مؤثر شناخته می‌شود.

روی بردارهای TF-IDF ، سه مدل کلاسیک آموزش داده شد:

: Naive Bayes

احتمال هر کلاس را بر اساس فرض استقلال شرطی بین کلمات محاسبه می‌کند؛ سریع اما ساده‌انگارانه.

: Logistic Regression

یک مرز تصمیم خطی بین کلاس‌ها می‌آموزد و خروجی احتمالاتی طبیعی تولید می‌کند.

: Linear SVM

به‌دنبال ابرصفحه‌ای است که بیشترین حاشیه (Margin) را بین کلاس‌ها ایجاد کند؛ در این پروژه به‌عنوان بهترین مدل کلاسیک انتخاب شد.

۲.۳ از RNN ساده تا GRUو LSTM

شبکه‌های عصبی بازگشتی (RNN) برای داده‌ی دنباله‌ای مانند متن طراحی شده‌اند: در هر گام زمانی، شبکه علاوه بر ورودی جدید، یک بردار حالت پنهان از گام قبلی دریافت می‌کند. مشکل اصلی RNN ساده این است که در دنباله‌های طولانی، گرادیان در فرایند Backpropagation-Through-Time یا به‌شدت کوچک می‌شود (Vanishing Gradient) و یادگیری متوقف می‌شود، یا بی‌نهایت بزرگ می‌شود (Exploding Gradient)و آموزش ناپایدار می‌گردد؛ در نتیجه شبکه در یادگیری وابستگی‌های بلندمدت (کلمات ابتدای جمله‌ای طولانی) ناتوان است.

شبکه‌ی LSTM (Long Short-Term Memory) این مشکل را با افزودن یک حافظه‌ی جداگانه به نام Cell Stateو سه دروازه (Gate) حل می‌کند: دروازه‌ی فراموشی (Forget Gate) تعیین می‌کند چه بخشی از حافظه‌ی قبلی کنار گذاشته شود؛ دروازه‌ی ورودی (Input Gate) تعیین می‌کند چه اطلاعات جدیدی به حافظه اضافه شود؛ و دروازه‌ی خروجی (Output Gate) تعیین می‌کند چه بخشی از حافظه

به‌عنوان خروجی آن گام زمانی ارائه شود. این مکانیزم به شبکه اجازه می‌دهد اطلاعات مهم را برای مدت طولانی‌تری نسبت به RNN ساده حفظ کند.

شبکه‌ی GRU (Gated Recurrent Unit) نسخه‌ی ساده‌ترشده‌ی LSTM است که تنها دو دروازه دارد (Update Gate و Reset Gate) و حافظه‌ی جداگانه‌ای (Cell State) ندارد؛ در عوض حالت پنهان به‌طور مستقیم نقش حافظه را ایفا می‌کند .پارامترهای کمتری نسبت به LSTM دارد و بنابراین سریع‌تر آموزش می‌بیند؛ روی دیتاست‌های کوچک‌تر معمولاً عملکردی قابل‌مقایسه یا حتی بهتر از LSTM دارد که در آزمایش‌های این پروژه (فصل پنجم) نیز مشاهده شد.

در این پروژه از نسخه‌ی دوطرفه (Bidirectional) این شبکه‌ها استفاده شده است؛ یعنی هر جمله هم‌زمان هم از چپ به راست و هم از راست به چپ پردازش می‌شود تا شبکه بتواند از Context هر دو طرف یک کلمه استفاده کند.

۲.۴ نرخ یادگیری، تابع هزینه و بهینه‌ساز

آموزش یک شبکه‌ی عصبی به‌معنای یافتن مقادیر وزن‌هایی است که تابع هزینه (Loss Function) را کمینه کند. برای مسئله‌ی طبقه‌بندی چندکلاسه‌ای مانند این پروژه، از تابع Categorical Cross-Entropy استفاده می‌شود (در این پروژه به‌شکل Sparse Categorical Cross-Entropy ، چون برچسب‌ها به‌صورت عدد صحیح و نه بردار One-Hot کدگذاری شده‌اند) .این تابع فاصله‌ی بین توزیع احتمال پیش‌بینی‌شده توسط مدل و برچسب واقعی را اندازه می‌گیرد؛ هرچه پیش‌بینی مدل به برچسب درست نزدیک‌تر باشد، مقدار این تابع کوچک‌تر است.
بهینه‌ساز (Optimizer) الگوریتمی است که با استفاده از گرادیان تابع هزینه، وزن‌های شبکه را در جهت کاهش خطا به‌روزرسانی می‌کند. در این پروژه از بهینه‌ساز Adam استفاده شده که ترکیبی از دو ایده است : Momentum(استفاده از میانگین متحرک گرادیان‌های قبلی برای حرکت پایدارتر) و نرخ یادگیری تطبیقی برای هر پارامتر به‌طور جداگانه. این ویژگی‌ها Adam را به انتخابی رایج و مؤثر برای اکثر مسائل یادگیری عمیق از جمله پردازش زبان طبیعی تبدیل کرده است.

نرخ یادگیری (Learning Rate) اندازه‌ی گامی است که بهینه‌ساز در هر به‌روزرسانی برمی‌دارد. نرخ یادگیری بیش از حد بزرگ باعث نوسان و عدم همگرایی مدل می‌شود، در حالی که نرخ یادگیری بیش از حد کوچک باعث کند شدن بیش از حد آموزش یا گیر افتادن در یک کمینه‌ی محلی نامطلوب می‌شود. در این پروژه مقدار ۰.۰۰۱ (پیش‌فرض رایج برایAdam) استفاده شده که نتایج پایداری به همراه داشت.

۲.۵ پردازش زبان طبیعی فارسی

پیش از ورود متن به هر مدلی، باید پیش‌پردازش شود. برای زبان فارسی این مرحله چالش‌های خاصی دارد: یکسان‌سازی کاراکترهای عربی و فارسی که ظاهری مشابه دارند ولی از نظر کد یونیکد متفاوتند (مانند «ی» عربی که دوتا نقطه زیر آن دارد در برابر «ی» فارسی)، مدیریت نیم‌فاصله که در بسیاری از متون کاربران به‌درستی استفاده نمی‌شود، و توکنایز کردن که به دلیل نبود فاصله‌گذاری استاندارد در برخی متون محاوره‌ای برای نرمال‌سازی و توکنایز کردن Hazm دشوارتر از انگلیسی است. در این پروژه از کتابخانه‌ی تخصصی .استفاده شده است

پس از توکنایز کردن، معمولاً استاپ‌وردها (کلمات پرتکرار و کم‌بار معنایی مانند «و»، «را»، «که») حذف می‌شوند تا مدل روی کلمات معنادارتر تمرکز کند. در نهایت، برای مدل‌های عمیق، هر کلمه باید به یک بردار قابل‌آموزش استفاده شده Embedding نگاشت شود؛ در این پروژه از یک لایه‌ی (Embedding) عددی که هم‌زمان با بقیه‌ی شبکه آموزش می‌بینند.

۳. دیتاست‌ها

۳.۱ (دیتاست اصلی) ArmanEmo

دیتاست اصلی این پروژه ArmanEmo است که توسط میرزایی و همکاران (۲۰۲۲) منتشر شده و به‌صورت عمومی در گیت‌هاب در دسترس است. این دیتاست شامل بیش از هفت هزار جمله‌ی فارسی جمع‌آوری‌شده از توییتر، اینستاگرام و بخش نظرات دیجی‌کالا است که هرکدام توسط برچسب‌گذاران انسانی به یکی از هفت دسته‌ی خشم، ترس، شادی، نفرت، غم، تعجب یا سایر برچسب خورده‌اند. جدول زیر آمار کلی این دیتاست را نشان می‌دهد:

مجموعه	تعداد نمونه
آموزش (train)	۶٬۱۵۷
آزمون (test)	۱٬۱۵۱
جمع کل	۷٬۳۰۸

توزیع کلاس‌ها در مجموعه‌ی آموزش)به‌ترتیب فراوانی (به شرح زیر است:

کلاس	تعداد نمونه	درصد
سایر	۱٬۶۸۶	۲۷.۴٪
خشم	۹۲۶	۱۵.۰٪
غم	۹۰۳	۱۴.۷٪
ترس	۷۵۹	۱۲.۳٪
تعجب	۷۴۱	۱۲.۰٪
شادی	۶۲۶	۱۰.۲٪
نفرت	۵۱۶	۸.۴٪

همان‌طور که در جدول بالا مشخص است، توزیع کلاس‌ها کاملاً نامتوازن است؛ کلاس «سایر» تقریباً سه برابر کلاس «نفرت» نمونه دارد. این نکته دلیل اصلی استفاده از معیار F۱-Macro و همچنین استفاده از پارامتر class_weight="balanced"هنگام آموزش مدل‌ها در این پروژه است.

۳.۲ (دیتاست مکمل برای آزمایش افزایش داده) EmoPars

EmoPars که توسط صبری و همکاران (۲۰۲۱) منتشر شده، شامل سی‌هزار توییت فارسی است. برخلاف ArmanEmo، هر نمونه در این دیتاست یک برچسب قطعی ندارد؛ بلکه برای هر یک از شش احساس پایه (خشم، ترس، شادی، نفرت، غم، تعجب) یک عدد رأی بین صفر تا پنج ثبت شده که نشان‌دهنده‌ی میزان توافق برچسب‌گذاران روی حضور آن احساس در متن است.

برای استفاده از این دیتاست در کنارArmanEmo ، لازم بود این رأی‌ها به یک برچسب قطعی تبدیل شوند. در این پروژه فقط نمونه‌هایی نگه داشته شدند که یک احساس به‌وضوح غالب بود: حداقل رأی احساس غالب باید سه (از پنج) باشد و فاصله‌ی آن با دومین احساس پررأی حداقل یک باشد. با این معیار، از میان سی‌هزار نمونه‌ی خام، ۵,۴۷۷ نمونه‌ی «باکیفیت» انتخاب شد.

۳.۳ پاک‌سازی و پیش‌پردازش داده

پیش از تبدیل متن به بردار عددی، مراحل زیر روی هر جمله انجام می‌شود:

۱. حذف لینک، منشن (@) و ایموجی، و تبدیل هشتگ به کلمه‌ی ساده

۲. کاهش تکرار غیرعادی حروف (مثلاً «خیلیییی» به «خیلی»)

۳. نرمال‌سازی حروف عربی/فارسی و نیم‌فاصله با استفاده از کتابخانه‌ی Hazm

٤. توکنایز کردن جمله به کلمات با توکنایزر Hazm

٥. حذف استاپ‌وردهای فارسی (فقط برای مسیر مدل کلاسیک؛ برای مدل عمیق استاپ‌وردها حفظ می‌شوند چون می‌توانند به معنای جمله کمک کنند)

۴. روش‌شناسی و پیاده‌سازی

۴.۱ معماری کلی سامانه

سامانه از شش مرحله‌ی اصلی تشکیل شده: (۱) جمع‌آوری و آماده‌سازی دیتاست، (۲) پیش‌پردازش متن فارسی، (۳) آموزش و ارزیابی مدل‌های ماشین لرنینگ کلاسیک، (۴) آموزش و ارزیابی مدل‌های یادگیری عمیق، (۵) ارزیابی و مقایسه‌ی نهایی مدل‌ها، و (۶) رابط کاربری Streamlit که هر دو مدل (کلاسیک و عمیق) را هم‌زمان به کاربر نمایش می‌دهد.

۴.۲ مدل‌های کلاسیک

برای استخراج ویژگی، به‌جای یک TfidfVectorizer ساده، از ترکیب دو وکتورایزر با FeatureUnion استفاده شد (این پیکربندی نهایی نتیجه‌ی آزمایش چهارم در فصل بعد است):

- **وکتورایزر سطح کلمه:** حداکثر ۲۰,۰۰۰ ویژگی، ngram_range=(۱, ۲) (تک‌کلمه و دوکلمه‌ای متوالی)، min_df=۲
- **وکتورایزر سطح کاراکتر:** حداکثر ۲۰,۰۰۰ ویژگی، analyzer="char_wb"، ngram_range=(۲, ۴) (زیررشته‌های ۲ تا ۴ کاراکتری در محدوده‌ی هر کلمه)
- هر دو وکتورایزر با sublinear_tf=True (استفاده از لگاریتم فراوانی به‌جای شمارش خطی) تنظیم شدند.

سه کلاسیفایر Naive Bayes، Logistic Regression و Linear SVM روی این بردارهای ترکیبی آموزش دیدند. از آنجا که LinearSVC به‌طور پیش‌فرض خروجی احتمالاتی (predict_proba) تولید نمی‌کند، برای امکان نمایش درصد اطمینان در رابط کاربری، نسخه‌ی نهاییِ بهترین مدل با CalibratedClassifierCV (روش Sigmoid، سه‌لایه اعتبارسنجی متقاطع) کالیبره شد.

۴.۳ مدل‌های یادگیری عمیق

معماری نهایی مدل عمیق)پس از یک دور بهینه‌سازی که در فصل بعد شرح داده می‌شود (به شکل زیر است:

```
Embedding(vocab_size=15000, output_dim=100, mask_zero=True)
SpatialDropout1D(0.3)
Bidirectional(LSTM|GRU, units=48)
Dropout(0.4)
Dense(32, activation='relu', kernel_regularizer=l2(1e-4))
Dropout(0.3)
Dense(7, activation='softmax')
```

پارامترهای آموزش: بهینه‌ساز Adam با نرخ یادگیری ۰.۰۰۱، تابع هزینه‌ی Sparse Categorical Cross-Entropy، اندازه‌ی Batch برابر ۶۴، حداکثر ۱۲ Epoch با EarlyStopping پایش val_loss، صبر = ۳ Epoch، بازگرداندن بهترین وزن‌ها)، و class_weight متوازن‌شده برای جبران عدم توازن کلاس‌ها. برای هر دو نوع سلول بازگشتی (LSTM و GRU) این معماری به‌طور جداگانه آموزش داده و مقایسه شد.

۴.۴ رابط کاربری

رابط کاربری با فریم‌ورک Streamlit و برند اختصاصی «Nyko» پیاده‌سازی شده است. کاربر متن فارسی خود را در یک کادر متنی وارد می‌کند و پس از فشردن دکمه‌ی «تحلیل کن»، دو کارت کنار هم نمایش داده می‌شود: نتیجه‌ی مدل آماری (کلاسیک) و نتیجه‌ی مدل عصبی (عمیق)، هرکدام به‌همراه یک نمودار میله‌ای افقی سفارشی (با Altair) از احتمال هر یک از هفت کلاس. این طراحی عمداً هر دو رویکرد را کنار هم نشان می‌دهد تا تفاوت عملکرد آن‌ها برای کاربر ملموس باشد.
به‌جای شکلک‌های صورتِ رایج(Emoji) ، هر یک از هفت کلاس احساس یک رنگ اختصاصی ثابت دارد که هم در کارت نتیجه و هم در نمودار استفاده می‌شود؛ این انتخاب طراحی، ظاهری مینیمال‌تر و حرفه‌ای‌تر به رابط کاربری می‌دهد. یک دکمه‌ی اطلاعات (ⓘ) در گوشه‌ی صفحه نیز اطلاعات نگارنده، استاد راهنما و دیتاست آموزشی را در قالب یک پنجره‌ی کوچک (Popover) در دسترس قرار می‌دهد.

۵. آزمایش‌ها و نتایج

۵.۱ نتایج پایه :مدل‌های کلاسیک در برابر مدل‌های عمیق

جدول زیر عملکرد نهایی همه‌ی مدل‌ها را روی مجموعه‌ی آزمون ArmanEmo نشان می‌دهد (مقادیر مدل‌های کلاسیک مربوط به پیکربندی نهایی، شامل ویژگی سطح کاراکتر، هستند؛ جزئیات در آزمایش چهارم):

مدل	داده‌ی آموزش	Accuracy	F۱-macro
Naive Bayes	ArmanEmo	۲۸.۳٪	۲۴.۶٪
Logistic Regression (بهترین کلی)	ArmanEmo	۵۲.۹٪	۵۰.۶٪
Linear SVM	ArmanEmo	۵۰.۵٪	۴۹.۳٪
Bi-LSTM	ArmanEmo	۳۸.۶٪	۳۶.۷٪
Bi-GRU (بهترین عمیق)	ArmanEmo	۳۷.۶٪	۳۷.۳٪

نتیجه‌ی قابل‌توجه این جدول آن است که مدل کلاسیک (Logistic Regression) با اختلاف زیادی (حدود ۱۳ واحد درصد در F۱-Macro) از بهترین مدل عمیق پیشی گرفته است؛ نتیجه‌ای که در نگاه اول با این باور رایج که «شبکه‌های عصبی همیشه بهترند» در تضاد است. در ادامه، چهار آزمایش کنترل‌شده برای بررسی و بهبود این نتیجه گزارش می‌شود.

۵.۲ آزمایش اول :بررسی علت ضعف مدل عمیق اولیه

در اولین تلاش، یک معماری دولایه (دو لایه‌ی Bidirectional LSTM/GRU پشت‌سرهم) با Embedding۱۲۸بعدی روی ArmanEmo آموزش داده شد. بررسی منحنی آموزش نشان داد که Accuracyروی داده‌ی آموزش به سرعت به بالای ۹۰٪ رسید، در حالی که val_loss از Epoch دوم یا سوم به بعد شروع به افزایش کرد. این الگو نشانه‌ی مشخص بیش‌برازش است: مدل به‌جای یادگیری الگوی کلی احساس، جزئیات نمونه‌های آموزش را حفظ کرده است. با ۶,۱۵۷ نمونه‌ی آموزش، یک لایه‌ی Embeddingکه کاملاً از صفر آموزش می‌بیند فرصت کافی برای یادگیری بازنمایی معنایی مناسب کلمات را ندارد.

۵.۳ آزمایش دوم: آیا افزایش داده با EmoPars کمک می‌کند؟

فرضیه: با توجه به کوچک بودنArmanEmo ، افزودن نمونه‌های باکیفیت EmoPars (که حجم داده‌ی آموزش را تقریباً دوبرابر می‌کند) باید بیش‌برازش را کاهش داده و عملکرد را بهبود بخشد.

مدل	داده‌ی آموزش	F۱-macro
Naive Bayes	ArmanEmo + EmoPars	۳۰.۹٪
Logistic Regression	ArmanEmo + EmoPars	۴۶.۸٪
Linear SVM	ArmanEmo + EmoPars	۴۲.۰٪
Bi-LSTM	ArmanEmo + EmoPars	۳۴.۱٪
Bi-GRU	ArmanEmo + EmoPars	۲۸.۸٪

نتیجه برخلاف فرضیه‌ی اولیه بود F۱-macro: مدل SVM از ٪۴۷.۵ (فقطArmanEmo) به ٪۴۲.۰ کاهش یافت و مدل‌های عمیق نیز بهبودی نشان ندادند. علت اصلی این است که برچسب‌های EmoPars به‌صورت خودکار و از روی آستانه‌ی رأی‌گیری استخراج شده‌اند، نه توسط قضاوت مستقیم انسانی مانند ArmanEmo؛ بنابراین نویز برچسب بیشتری دارند و به‌جای کمک، سیگنال گمراه‌کننده به مدل منتقل می‌کنند. این آزمایش یک اصل شناخته‌شده در یادگیری ماشین را دوباره تأیید کرد: کیفیت داده اغلب مهم‌تر از حجم آن است.

نکته‌ی فنی: در حین این آزمایش یک اشکال پیاده‌سازی نیز کشف و رفع شد. از آنجا که دیتاست ترکیبی از چسباندن مستقیم ArmanEmo و EmoPars ساخته شده بود (بدون درهم‌ریزی تصادفی)، پارامتر validation_splitدر Keras که به‌طور پیش‌فرض داده‌ی اعتبارسنجی را از انتهای آرایه انتخاب می‌کند، عملاً یک مجموعه‌ی اعتبارسنجی کاملاً غیرنماینده (فقط ازEmoPars) می‌ساخت. این موضوع باعث می‌شد سیگنال val_lossاز همان ابتدا نامعتبر باشد و EarlyStopping خیلی زودتر از موعد مدل را متوقف کند. راه‌حل، درهم‌ریزی تصادفی (Shuffle) کامل داده‌ی ترکیبی پیش از آموزش بود.

۵٫٤ آزمایش سوم :آیا ساده‌سازی معماری کمک می‌کند؟

فرضیه: با توجه به حجم محدود داده، یک معماری کوچک‌تر با تنظیم‌کننده‌های قوی‌تر (Regularization)باید بهتر از یک معماری بزرگ عمل کند.

تغییرات اعمال‌شده نسبت به آزمایش اول: کاهش از دو لایه‌ی Bidirectional به یک لایه، کاهش بعد Embeddingاز ۱۲۸ به ۱۰۰، افزودن SpatialDropout۱D(که برخلاف Dropout معمولی، کل بردار Embedding یک کلمه را حذف می‌کند نه تک‌تک اعداد آن را — روشی که برای داده‌ی متنی مؤثرتر شناخته می‌شود)، و افزودن L۲ Regularization به لایه‌ی Dense.

نتیجه (روی ArmanEmo تنها، بدونEmoPars): F۱-macro مدل GRU به ٪۳۷.۳ و LSTM به ٪۳۶.۷ رسید — بهبودی نسبت به معماری اولیه، هرچند هنوز به سطح مدل کلاسیک نرسید. لازم به ذکر است که در طول این آزمایش‌ها، نوسان قابل‌توجهی) تا حدود پنج واحد درصد در (F۱-macro بین اجراهای مختلف مشاهده شد که به دلیل مقداردهی تصادفی وزن‌های اولیه‌ی شبکه بود؛ این مسئله با ثابت کردن Seed رندوم (tf.random.set_seed) در نسخه‌ی نهایی کد رفع شد تا نتایج تکرارپذیر باشند.

۵.۵ آزمایش چهارم :آیا افزودن ویژگی سطح کاراکتر کمک می‌کند؟

این آزمایش با انگیزه‌ای متفاوت از سه آزمایش قبلی انجام شد: در تست دستی رابط کاربری، مشاهده شد جمله‌های محاوره‌ای و غیررسمی (مثل «حالم خیلی خوبه») هرچند به‌درستی طبقه‌بندی می‌شدند، درصد اطمینان نسبتاً پایینی می‌گرفتند. فرضیه: چون TF-IDF سطح کلمه به تغییرات املایی و نوشتار محاوره‌ای (که در استفاده‌ی واقعی بسیار بیشتر از متن نسبتاً رسمی دیتاست دیده می‌شود) حساس است، افزودن ویژگی سطح کاراکتر باید به مقاوم‌تر شدن مدل کمک کند.

تغییر: به‌جای یک TfidfVectorizer، از ترکیب یک وکتورایزر سطح کلمه (۱, ۲)=ngram_rangeو یک وکتورایزر سطح کاراکتر (۲, ٤)=analyzer="char_wb", ngram_range با FeatureUnionاستفاده شد، به‌همراه sublinear_tf=Trueروی هر دو.

مدل	ویژگی قبلی)فقط کلمه(	ویژگی جدید)کلمه+کاراکتر(
Linear SVM	۴۷.۵٪	۴۹.۳٪
Logistic Regression	۴۷.۰٪	۵۰.۶٪

نتیجه: فرضیه تأیید شد Logistic Regression .مدل F۱-macro به ۵۰.۶٪ رسید — بهترین نتیجه‌ی کل پروژه. n-gram های کاراکتری به تکه‌های زیرکلمه‌ای (شبیه ریشه‌ی فعل یا پسوند) حساس‌ترند و در برابر تغییرات نوشتاری محاوره‌ای مقاوم‌ترند. علاوه بر بهبود F۱-macro ، این تغییر مستقیماً مشکل اطمینان پایین را نیز بهبود داد؛ برای مثال اطمینان مدل روی جمله‌ی «حالم خیلی خوبه» از ۳۵.۹٪ به ۵۹.۶٪ افزایش یافت.

۵.۵ چرا درصد اطمینان مدل گاهی پایین به‌نظر می‌رسد؟

در جریان تست دستی رابط کاربری، این پرسش مطرح شد که چرا مدل روی جمله‌های به‌ظاهر ساده و واضح (از نظر احساسیِ گوینده)، درصد اطمینان بالایی نشان نمی‌دهد. بررسی این پدیده سه دلیل اصلی را روشن کرد که ارزش ثبت در گزارش را دارند:

۱. **مسئله‌ی این پروژه هفت‌کلاسه است؛** حدس کاملاً تصادفی حدود ۱۴٪ اطمینان می‌گیرد. بنابراین اطمینان ۵۰ تا ۶۰ درصدی که مدل نشان می‌دهد، سه تا چهار برابر بهتر از سطح شانس است، هرچند در نگاه اول عدد کوچکی به‌نظر می‌رسد.

۲. **کالیبراسیون درست یعنی صداقت آماری، نه اعتمادبه‌نفس بالا .**چون دقت کلی مدل حدود ۵۰ تا ۵۳ درصد است، یک مدل خوب‌کالیبره‌شده باید به‌طور میانگین همین‌قدر اطمینان نشان دهد؛ اگر مدل همیشه بالای ۹۰٪ اطمینان نشان می‌داد، این خود نشانه‌ی بیش‌اعتمادی (Overconfidence) و کالیبراسیون نادرست بود، نه مدل بهتر.

۳. **بین سبک متن‌های آزمایشی و سبک دیتاست آموزش عدم‌تطابق نسبی وجود دارد (Distribution Shift):** ArmanEmo از شبکه‌های اجتماعی با لحن معمولاً پرشور و صریح جمع‌آوری شده، در حالی که جمله‌های محاوره‌ای و آرام کلمات به‌همان‌اندازه تمایزدهنده ندارند.

همان‌طور که در آزمایش چهارم دیده شد، بخشی از این مشکل (مورد سوم) با افزودن ویژگی سطح کاراکتر واقعاً قابل بهبود بود؛ اما بخشی از آن (موارد اول و دوم) ماهیتی بنیادی دارد و نشان‌دهنده‌ی رفتار صادقانه‌ی یک مدل به‌درستی کالیبره‌شده است، نه یک نقص قابل رفع.

۵.۷ نمونه‌ی کیفی :کلمات خارج از دایره واژگان آموزشی

در تست دستی دیگری، جمله‌ی «کالاف رو دانلود کردم، هورا «!(اعلام هیجان‌زده‌ی یک بازی‌باز از دانلود بازی Call of Duty) به هر دو مدل داده شد. نتیجه برخلاف انتظار (احساس شادی) بود: مدل آماری «سایر» را با ٪۵۴.۶ اطمینان و مدل عصبی «غم» را با تنها ٪۲۴.۴ اطمینان (یعنی حتی از سطح شانس تصادفی هم چندان بالاتر نه) پیش‌بینی کردند.

علت این خطا روشن است: «کالاف» یک وام‌واژه‌ی محاوره‌ای رایج در فرهنگ بازی‌های ویدیویی است که در دیتاست ArmanEmo (جمع‌آوری‌شده از موضوعات عمومی توییتر، اینستاگرام و دیجی‌کالا) به‌احتمال زیاد اصلاً حضور ندارد؛ «هورا» نیز یک شبه‌جمله‌ی نشان‌دهنده‌ی شادی است که در پیکره‌ی آموزشی کم‌تکرار است. وقتی مهم‌ترین کلمات یک جمله از نظر مدل ناشناخته یا کم‌تکرار باشند (خارج از دایره‌ی واژگانی که مدل در آموزش دیده، پدیده‌ای که در ادبیات یادگیری ماشین Out-of-Vocabulary یا به‌طور کلی‌تر Distribution Shift نامیده می‌شود)، سیگنال قابل‌اتکایی برای مدل باقی نمی‌ماند و پیش‌بینی به‌جای یک پیک واضح، در میان چند کلاس پخش می‌شود.

این نمونه به‌خوبی نشان می‌دهد یک محدودیت بنیادی و شناخته‌شده‌ی هر مدل یادگیری نظارت‌شده را: کیفیت پیش‌بینی برای دنیای واقعی، به میزان همپوشانی واژگان و سبک نوشتاری ورودی با دیتاست آموزشی وابسته است. این محدودیت مختص این پروژه نیست و در عمل تقریباً هر سامانه‌ی NLP تجاری نیز با آن مواجه است؛ راهکار معمول آن، پایش مستمر و افزودن تدریجی داده‌های آموزشی از دامنه‌های جدید (مانند اسلنگ و اصطلاحات تخصصی) است.

جمع‌بندی آزمایش‌ها ۵.۸

پیکربندی	F۱-macro بهترین
تنها ArmanEmo — (کلمه+کاراکتر ،LogReg) مدل کلاسیک	٪۵۰.۶ (بهترین کلی)
تنها ArmanEmo — (فقط کلمه ،SVM) مدل کلاسیک	٪۴۷.۵
تنها ArmanEmo — مدل عمیق)معماری ساده‌شده(	٪۳۷.۳
تنها ArmanEmo — مدل عمیق)معماری اولیه، دولایه(	٪۳۸.۲
ArmanEmo + EmoPars — (SVM) مدل کلاسیک	٪۴۲.۰
ArmanEmo + EmoPars — مدل عمیق)معماری ساده‌شده(	٪۳۴.۱

۶. بحث و تحلیل

نتایج این پروژه نشان می‌دهد که برتری مدل‌های یادگیری عمیق نسبت به روش‌های کلاسیک، برخلاف تصور رایج، به‌هیچ‌وجه تضمین‌شده نیست و شدیداً به حجم و کیفیت داده‌ی در دسترس وابسته است. مدل‌های عمیقی که در این پروژه استفاده شدند، یک لایه‌ی Embedding را کاملاً از صفر آموزش می‌دهند؛ یعنی مدل باید هم‌زمان با یادگیری وظیفه‌ی طبقه‌بندی، معنای هر کلمه را هم از ابتدا کشف کند. با کمتر از هفت هزار جمله‌ی آموزشی، این کار برای شبکه‌ای با میلیون‌ها پارامتر بسیار دشوار است و نتیجه‌ی آن بیش‌برازش زودهنگام است. در مقابل، TF-IDF یک بازنمایی آماری ثابت و مبتنی بر کل دیتاست است که پارامتر قابل‌آموزشی برای هر کلمه ندارد؛ به همین دلیل کمتر مستعد بیش‌برازش روی دیتاست‌های کوچک است. این یافته با نتایج مشابه در ادبیات پردازش زبان طبیعی همخوانی دارد: مدل‌های عمیق معمولاً زمانی برتری چشمگیر خود را نشان می‌دهند که یا حجم داده بسیار زیاد باشد، یا از وزن‌های از پیش‌آموزش‌دیده (مانند Embedding های آماده یا مدل‌های زبانی بزرگ مثل BERT) استفاده شود — که هیچ‌کدام در این پروژه به دلیل محدودیت زمان و دسترسی امکان‌پذیر نبود.

آزمایش افزایش داده با EmoPars نیز نکته‌ی مهم دیگری را روشن کرد: افزایش صرف حجم داده، بدون توجه به کیفیت برچسب‌ها، می‌تواند به عملکرد مدل آسیب بزند. برچسب‌های استخراج‌شده از رأی‌گیری آستانه‌ای EmoPars قطعیت کمتری نسبت به برچسب‌گذاری مستقیم انسانی ArmanEmo دارند و این نویز اضافه، سیگنال آموزشی مدل را ضعیف‌تر می‌کند تا اینکه آن را تقویت کند.

از منظر عملی، این یافته‌ها به این معنا نیست که مدل کلاسیک همیشه انتخاب بهتری است؛ بلکه نشان می‌دهد انتخاب معماری باید متناسب با مقیاس و کیفیت داده‌ی در دسترس باشد. برای این مسئله‌ی خاص با این حجم داده، رویکرد کلاسیک انتخاب عملی‌تری بود، اما با دیتاست بزرگ‌تر یا استفاده از مدل‌های زبانی از پیش‌آموزش‌دیده، انتظار می‌رود مدل‌های عمیق برتری خود را نشان دهند.

آزمایش چهارم (افزودن ویژگی سطح کاراکتر) نشان داد که حتی در چارچوب مدل‌های کلاسیک هم هنوز جای بهبود ساده و کم‌هزینه وجود دارد؛ گاهی مهندسی ویژگی (Feature Engineering) هوشمندانه، تاثیر بیشتری از تعویض کل الگوریتم دارد. همچنین تحلیل اطمینان مدل یادآوری مهمی است: در ارزیابی یک سامانه‌ی یادگیری ماشین، «مدل مطمئن‌تر» لزوماً به‌معنای «مدل بهتر» نیست؛ مدلی که به‌درستی کالیبره شده، اطمینانش را متناسب با دقت واقعی‌اش نشان می‌دهد، حتی اگر آن عدد در نگاه اول کوچک به‌نظر برسد.

۷. نتیجه‌گیری و کارهای آینده

در این پروژه یک سامانه‌ی کامل تشخیص احساس از روی متن فارسی طراحی و پیاده‌سازی شد که شامل پایپلاین پیش‌پردازش متن فارسی، دو رویکرد یادگیری ماشین (کلاسیک و عمیق)، و یک رابط کاربری تعاملی است. نتایج نشان داد که مدل کلاسیک Logistic Regression با ویژگی‌های ترکیبی کلمه و کاراکتر، با F۱-macro برابر ۵۰.۶٪ بهترین عملکرد را در میان همه‌ی مدل‌های آزمایش‌شده داشت و از مدل‌های عمیق LSTM و GRU (که بهترینشان ۳۷.۳٪ بود) به‌وضوح پیشی گرفت. با انجام چهار آزمایش کنترل‌شده، مشخص شد که علت ضعف نسبی مدل‌های عمیق، بیش‌برازش ناشی از حجم محدود داده است؛ افزایش حجم داده با برچسب‌های نویزی (EmoPars) این مشکل را حل نکرد، اما مهندسی ویژگی هوشمندانه‌تر برای مدل کلاسیک (افزودن ویژگی سطح کاراکتر) هم دقت را بهبود داد و هم اطمینان مدل را روی متن‌های محاوره‌ای واقعی افزایش داد.

به‌عنوان کارهای آینده، مسیرهای زیر پیشنهاد می‌شود:

استفاده از Embedding از پیش‌آموزش‌دیده‌ی فارسی :بکارگیری بردارهای آموخته‌شده مانند FastTextبه‌جای آموزش لایه‌ی Embedding از صفر.

Fine-tuneکردن مدل‌های زبانی بزرگ :بهره‌گیری از یک مدل پایه و از پیش‌آموزش‌دیده مانند ParsBERTبرای این وظیفه.

به‌بهود کیفیت دیتاستEmoPars : بازبینی یا برچسب‌گذاری دستی بخشی از EmoPars برای کاهش نویز برچسب پیش از استفاده درData Augmentation .

توسعه و پوشش دامنه‌ی واژگانی :جمع‌آوری داده‌های برچسب‌گذاری‌شده‌ی بیشتر، به‌ویژه برای کلاس‌های کم‌نمونه مانند «نفرت» و همچنین اصطلاحات تخصصی و اصطلاحات محاوره‌ای (اسلنگ).

ارزیابی کیفی و خطایابی ریزبینانه :بررسی معیارهای ارزیابی تکمیلی مانند ماتریس درهم‌ریختگی (Confusion Matrix)برای شناسایی الگوهای اشتباه مدل بین کلاس‌های نزدیک به هم (مثلاً ترس و تعجب).

منابع

[۱] Mirzaee, H., Peymanfard, J., Moshtaghin, H. Z., & Zeinali, H. (۲۰۲۲). ArmanEmo: A Persian Dataset for Text-based Emotion Detection. arXiv preprint.

[۲] Sabri, N., Akhavan, R., & Bahrak, B. (۲۰۲۱). EmoPars: A Collection of ۳۰K Emotion-Annotated Persian Social Media Texts.

[۳] Hochreiter, S., & Schmidhuber, J. (۱۹۹۷). Long Short-Term Memory. Neural Computation, ۹(۸), ۱۷۳۵-۱۷۸۰.

[٤] Cho, K., van Merrienboer, B., Gulcehre, C., et al. (۲۰۱۴). Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation. EMNLP.

[٥] Kingma, D. P., & Ba, J. (۲۰۱۵). Adam: A Method for Stochastic Optimization. ICLR.

[٦] Pedregosa, F., et al. (۲۰۱۱). Scikit-learn: Machine Learning in Python. JMLR, ۱۲, ۲۸۲۵-۲۸۳۰.

[۷] Abadi, M., et al. (۲۰۱۶). TensorFlow: A System for Large-Scale Machine Learning. OSDI.

[۸] کتابخانه‌ی hazm — ابزار پردازش زبان طبیعی فارسی. github.com/roshan-research/hazm